Automotive embedded system

Practice 04. CAN

Automotive Intelligence Lab.

Copyright © AI Lab. All rights reserved.

Contents

- Install Kvaser driver and CANlib using Python
- Receiving and Transmitting CAN messages in CAN bus

Copyright © AI Lab. All rights reserved.

CAN Practice

Copyright © AI Lab. All rights reserved.

Method of CAN Practice

Practice equipment

- Jetson AGX Orin Dev-Kit 64GB
  - Ubuntu 20.04 / Python 3.7+
- Kvaser Leaf v3



Method

1. Follow the tutorials in slide.
2. Ask questions about tutorials.
3. If you finish the chapter, submit your result.
   If you don’t, do it in your home and submit the result.
4. Go to next chapter and repeat the procedure.



Copyright © AI Lab. All rights reserved.

Today practice course

- **Chapter 1**
  - Install Kvaser driver and CANlib using Python
- **Chapter 2 ~ 3**
  - Receiving and Transmitting CAN messages in CAN bus

Copyright © AI Lab. All rights reserved.

Chapter 1

Install Kvaser driver and CANlib using Python

Copyright © AI Lab. All rights reserved.

Chapter 1

Backgrounds

- Install CAN Message driver
- Basic example of CAN transmitter

Notice

- It is necessary to connect Orin with Kvaser CAN device.
  - You need to connect Orin with Kvaser CAN device after the Kvaser driver is installed.

Goal

- Usage of the CANlib library to send CAN Message via Kvaser CAN device

Copyright © AI Lab. All rights reserved.

Install Kvaser Linux Driver

Install the driver

```
$ sudo apt-get install build-essential
$ sudo apt-get install pkg-config
$ wget --content-disposition "https://www.kvaser.com/downloads-
kvaser/?utm_source=software&utm_ean=7330130980754&utm_status=latest"
$ tar xvzf linuxcan.tar.gz
$ cd linuxcan
$ make
$ sudo make install
$ sudo make load
```

Install generic and remove unnecessary

```
$ sudo apt-get install linux-headers-generic
$ sudo apt remove dkms
```

Copyright © AI Lab. All rights reserved.

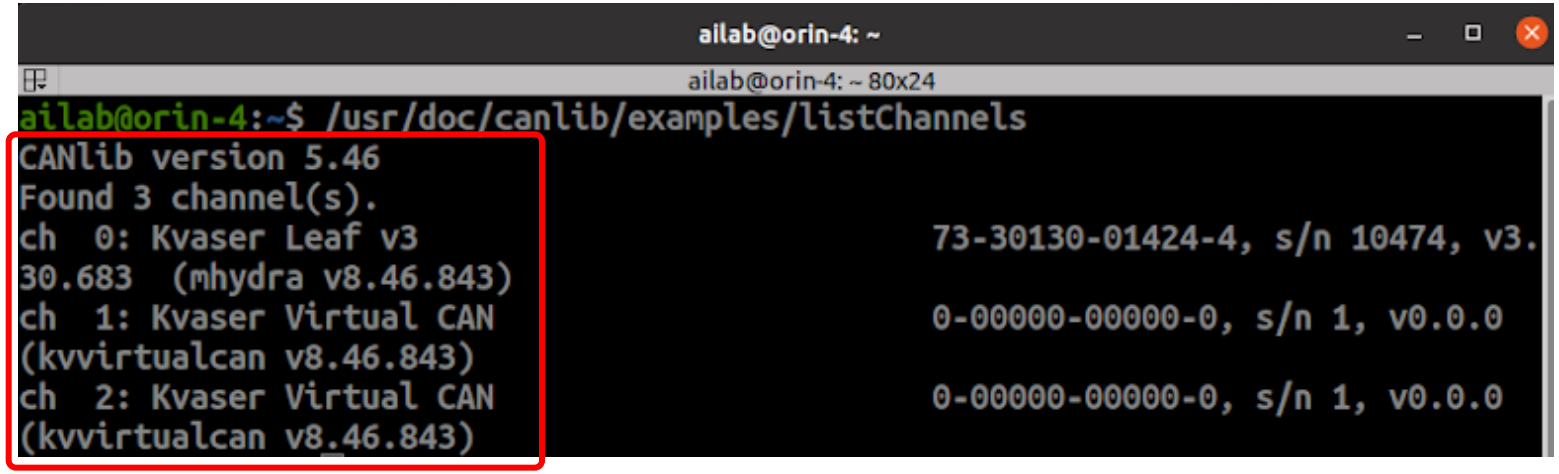
Install Kvaser Linux Driver

Installing Kvaser CANlib SDK

```
$ wget --content-disposition "https://www.kvaser.com/downloads-
kvaser/?utm_source=software&utm_ean=7330130981966&utm_status=latest"
$ tar xvzf kvlibsdk.tar.gz
$ cd kvlibsdk
$ make
$ sudo make install
```

정상 작동 여부 확인 (Kvaser Leaf v3 연결)

```
$ /usr/doc/canlib/examples/listChannels
```

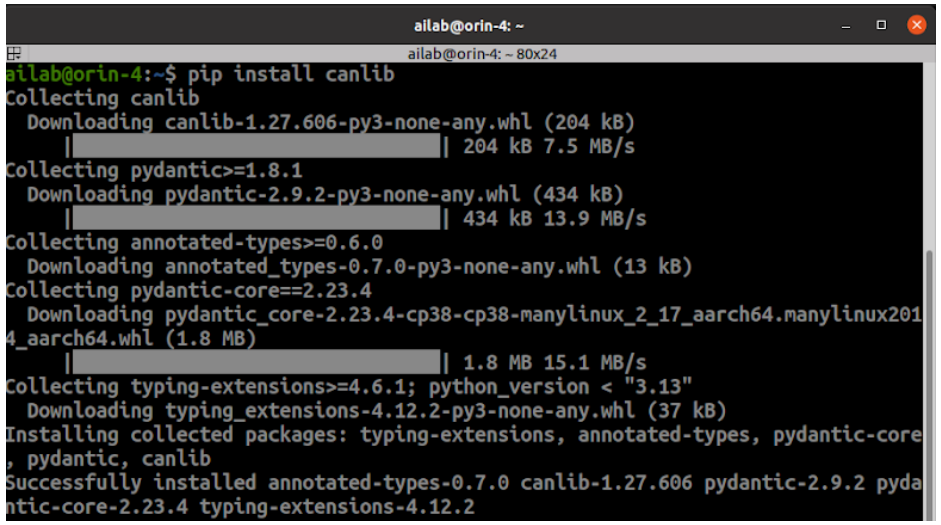


Copyright © AI Lab. All rights reserved.

Install Python CANlib Package

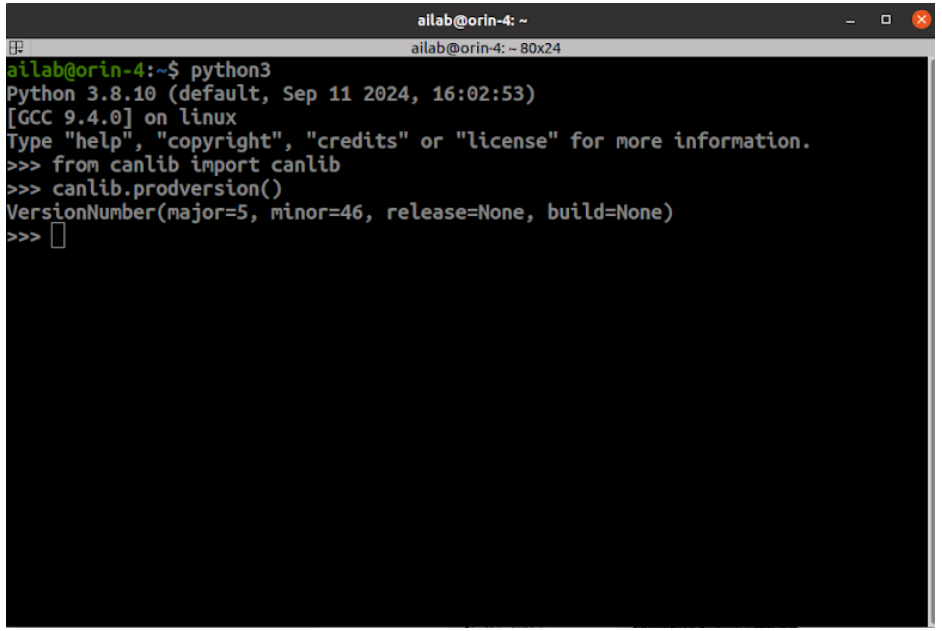
Install CANlib Python package

```
$ pip install canlib
```



Check if the CANlib is successfully installed

```
$ python3
>>> from canlib import canlib
>>> canlib.prodversion()
```



Copyright © AI Lab. All rights reserved.

Verify the Installation

Python example code

```python
# Firstly import canlib so that it can be used in the script.
from canlib import canlib

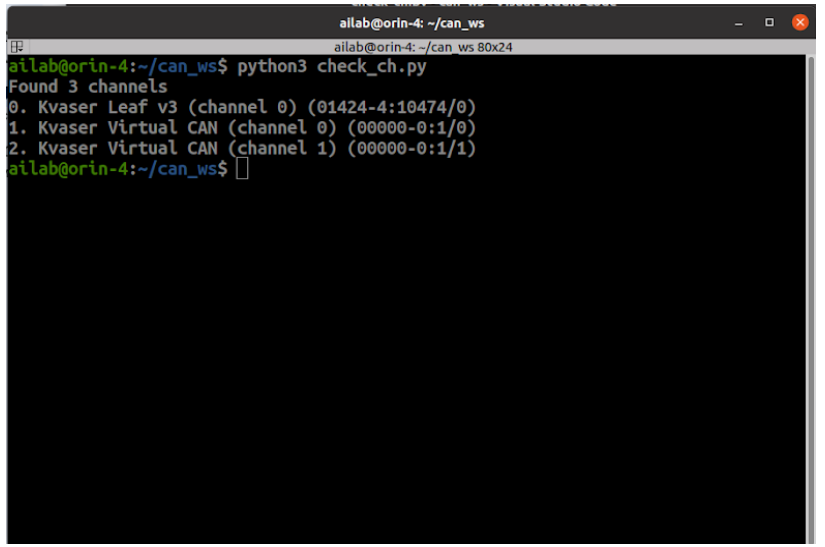
# .getNumberOfChannels() is used to detect the number of channels and
# the number is saved in the variable num_channels.
num_channels = canlib.getNumberOfChannels()

# num_channels is printed out as text so that the user can see how many
# channels were found.
print(f"Found {num_channels} channels")

# Next a for loop is created. This loop will repeat the code within for each
# channel that was detected.
for ch in range(num_channels):
# The data of each specific channel is saved in chd.
    chd = canlib.ChannelData(ch)
# Lastly the channel, channel name, product number, serial number, and local
# channel number on the device are printed.
    print(f"{ch}. {chd.channel_name} ({chd.card_upc_no.product()}:{chd.card_serial_no}/{chd.chan_no_on_card})")
```

Execute Python example code

```
$ python3 check_ch.py
```



Copyright © AI Lab. All rights reserved.

Send a CAN message

Send a CAN message

주석 읽어보면서 코드 작성

```
from canlib import canlib, Frame
import time

# Firstly, open CAN channel. In this example the channel is named ch_a.
# To open the channel call on the openChannel method inside of canlib and, as an
# input put in channel=0. Where 0 represents the CANlib channel 0.
ch_a = canlib.openChannel(channel=0)

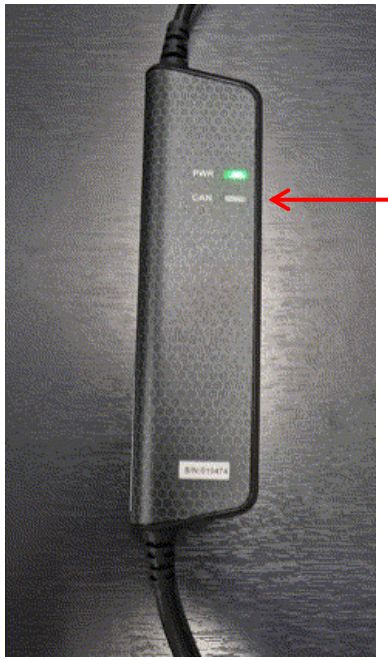
# After opening the channel, we need to set the bus parameters.
# For now we will use setBusParams() to set the canBitrate to 250K.
ch_a.setBusParams(canlib.canBITRATE_250K)
cnt = 1

while True:
    # The next step is to Activate the CAN chip for each channel (ch_a in this example)
    # use .busOn() to make them ready to receive and send messages.
    ch_a.busOn()

    # To transmit a message with (11-bit) CAN id = 123 and contents (decimal)
    # 72, 69, 76, 76, 79, 33, first create the CANFrame (CANmessage) and name it.
    # In this example, the CANFrame is named frame. Then send the message by calling on
    # the channel that will act as the sender and use .write() with the CANFrame
    # as input. In this example ch_a will act as sender.
    frame = Frame(id_=123, data=[72, 69, 76, 76, 79, 33], flags=canlib.MessageFlag.STD )
    ch_a.write(frame)

    print(f"[msg-{cnt}]")
    print("========================================\n [Message Sent!]")
    print(f" -ID:{frame.id}\n -DLC:{frame.dlc}\n -Data:{[byte for byte in frame.data]}")
    print("========================================\n")
    # After the message has been sent, received and read it is time to inactivate
    # the CAN chip. To do this call .busOff() on both channels that went .busOn()
    ch_a.busOff()
    cnt = cnt + 1
    time.sleep(1)

# Lastly, close all channels with close() to finish up.
ch_a.close()
```

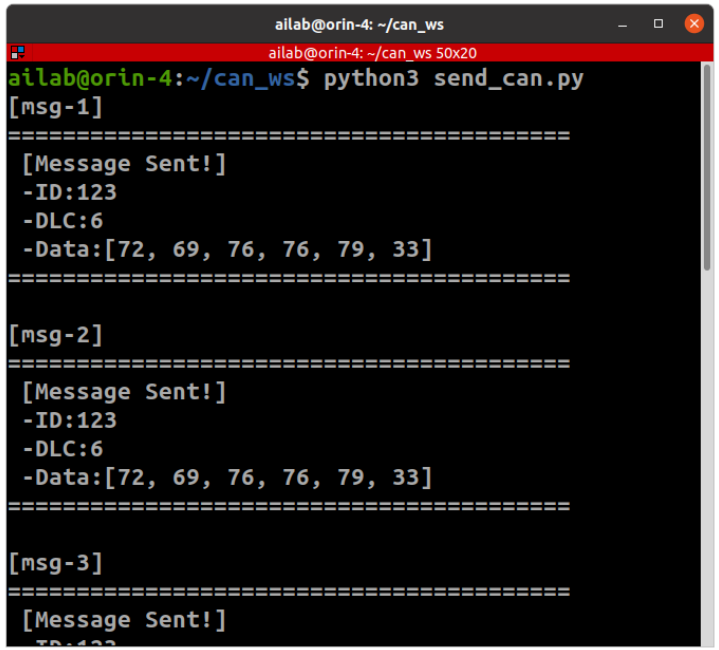


CAN 라이트가 깜빡이는 지 확인

Copyright © AI Lab. All rights reserved.

Chapter1

Result of the Practice



Show T.A. that your Kvaser is working!

Copyright © AI Lab. All rights reserved.

Chapter 2

Receiving and Transmitting CAN messages in CAN bus

Copyright © AI Lab. All rights reserved.

Chapter 2

Backgrounds

- CAN communication practice based on Chapter1.
- Creating CAN bus and receive & transmit CAN data.

Notice

- You must have finished Chapter1 to do this practice.
- Use team Orin and Kvaser device for this practice.

Goal

- Receive CAN data from T.A. and transmit your own data.

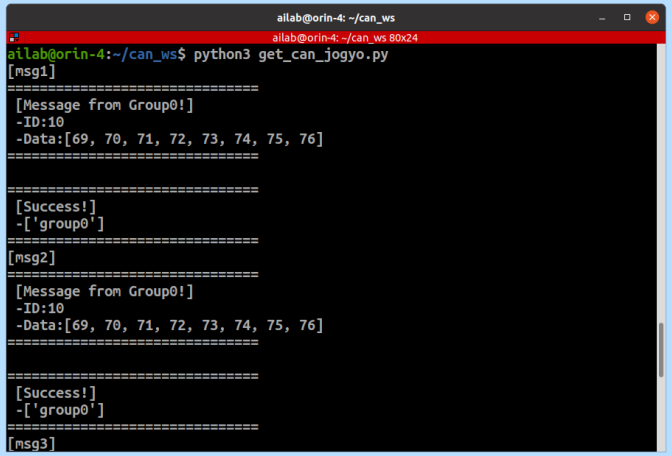
Copyright © AI Lab. All rights reserved.

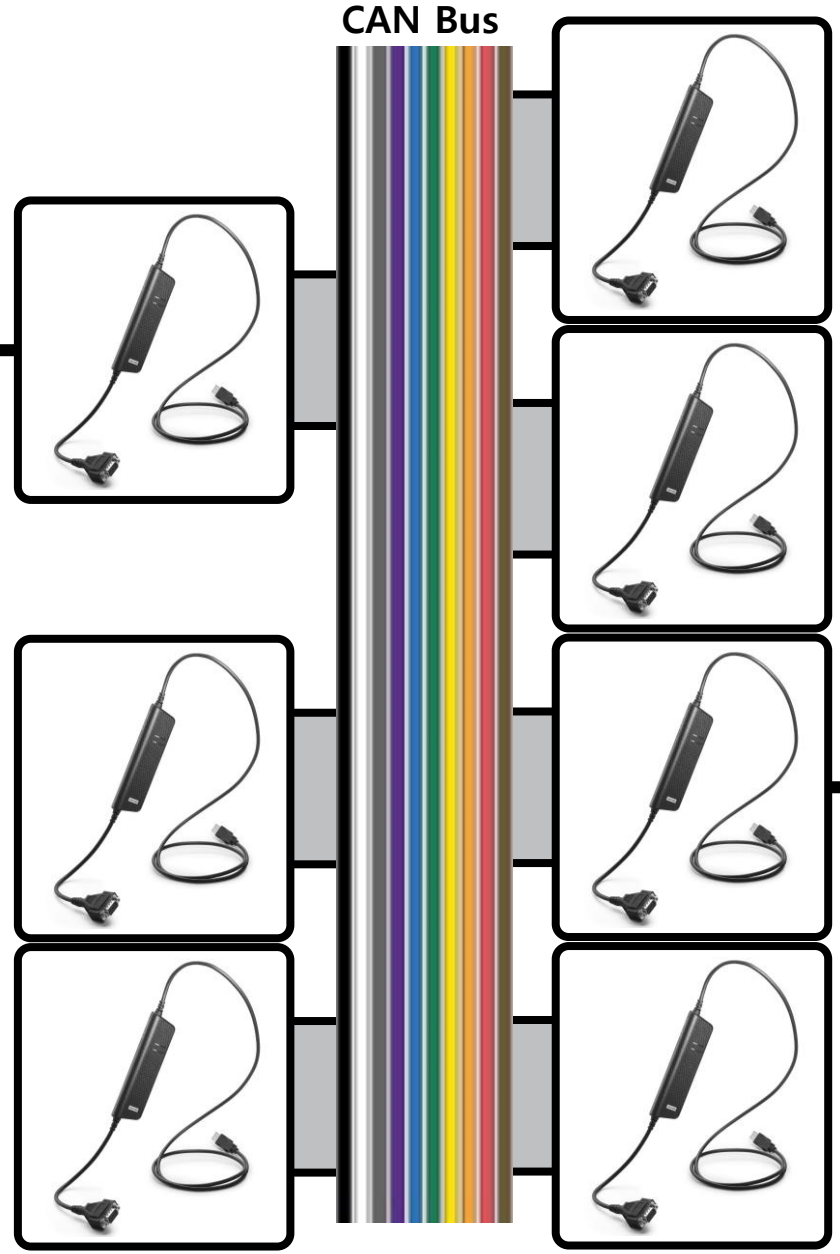
Chapter 2

Create a CAN Bus

T.A.

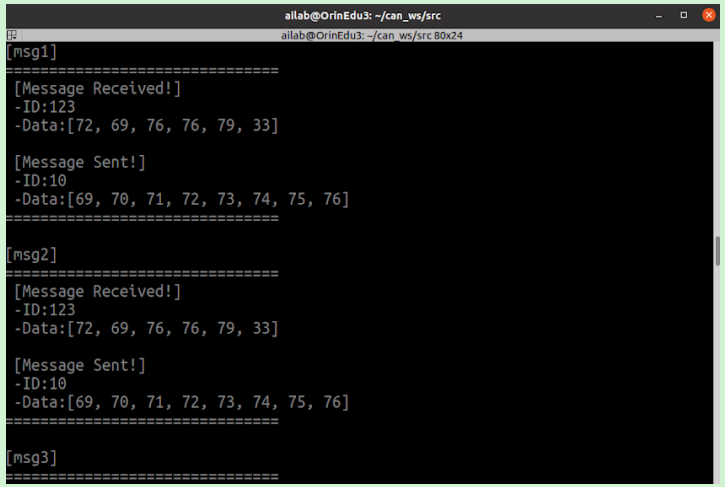
- Transmit dummy data
- Monitoring students' data





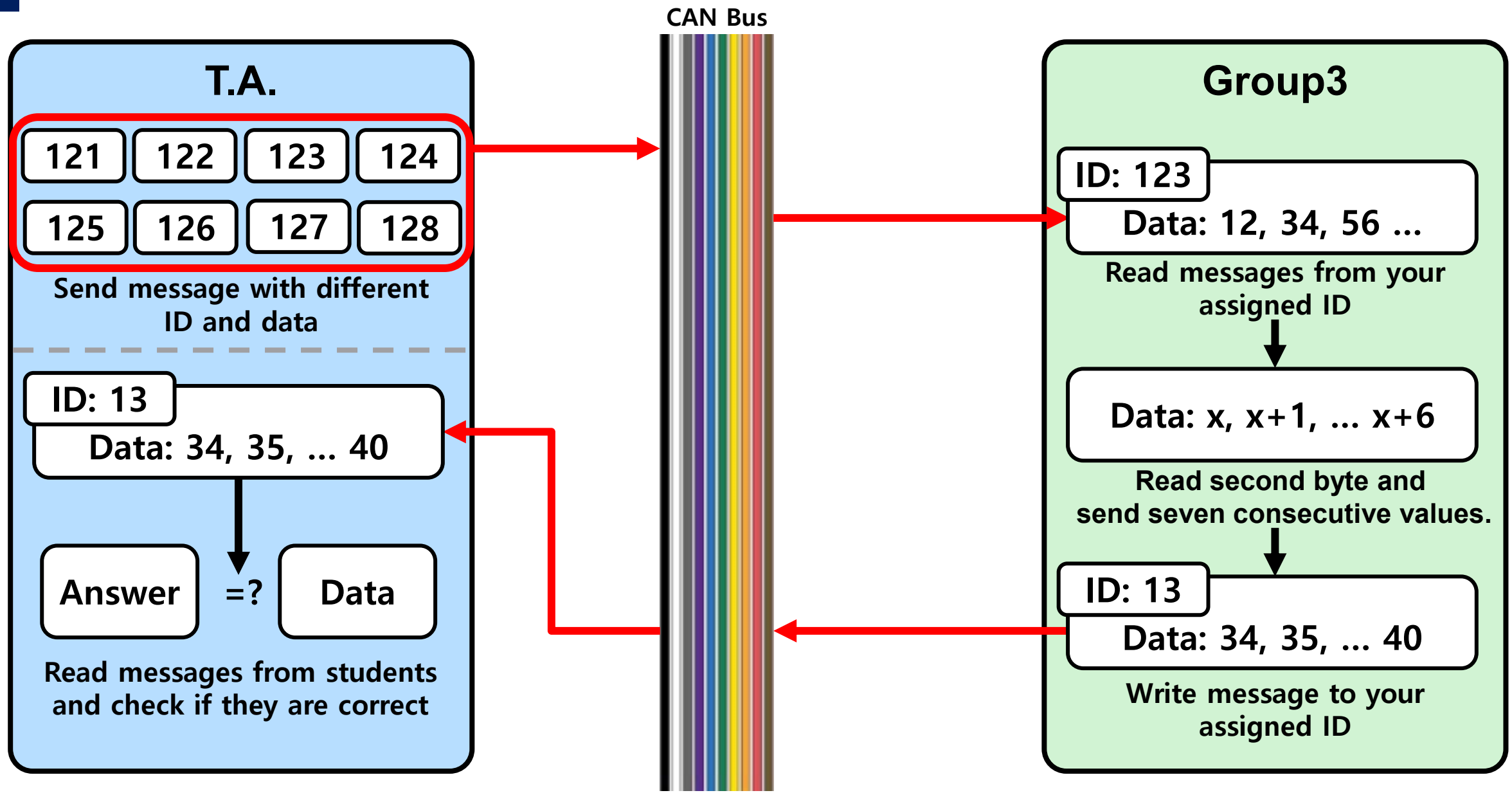
Students

- Receive T.A. data
- Processing data
- Transmit processed data



Copyright © AI Lab. All rights reserved.

Architecture



Copyright © AI Lab. All rights reserved.

Receive & Transmit

Receive and Transmit CAN data!

```python
from canlib import canlib, Frame


# Setting CAN bus
ch_a = canlib.openChannel(channel=0)
ch_a.setBusParams(canlib.canBITRATE_250K)
ch_a.busOn()


cntr = 1
while True:
    try:
        msg = ch_a.read(timeout=500)
        if msg.id == 123:
            print(f"[msg{cntr}]")
            print(f"==============================\n [Message Received!]")
            print(f" -ID:{msg.id}\n -Data:{[byte for byte in msg.data]} \n" )
            # TODO: Get specific field data from the message
            # In this case, the data is the second byte of the message

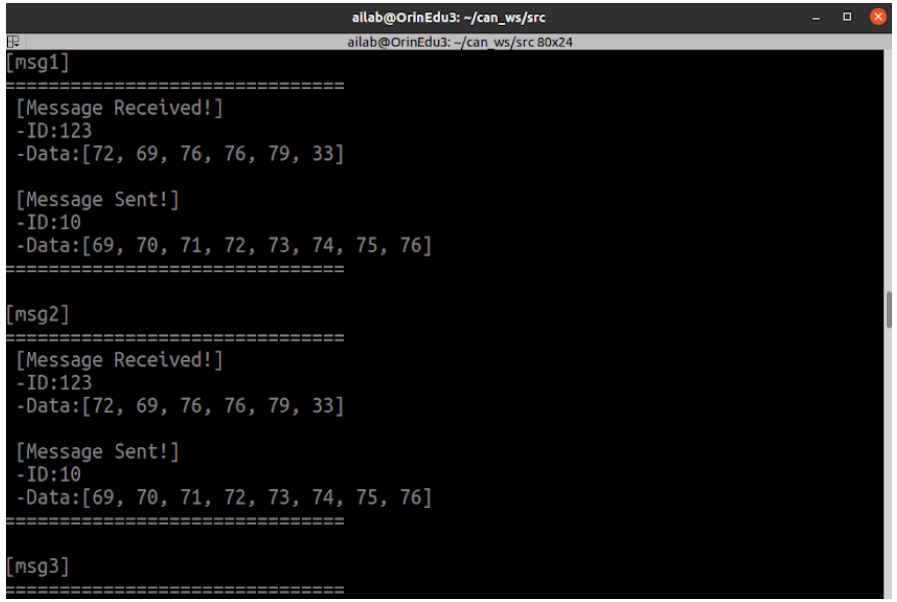
            # TODO: Update specific id
            # In this case, the id is the same byte as the message

            # TODO: Send message

            print(" [Message Sent!]")
            print(f" -ID:{frame.id}\n -Data:{[byte for byte in frame.data]} \n==============================\n")
            cntr = cntr + 1

    except canlib.canNoMsg:
        # waiting message
        pass


# Shutting off CAN bus
ch_a.busOff()
ch_a.close()
```



Copyright © AI Lab. All rights reserved.

Check your result

Receive and Transmit CAN data!

Group	Group Members	Read	Write
1	김정태, 임정기, 김동진	121	11
2	박수빈, 정다훈	122	12
3	허승우, 김석재	123	13
4	김다혜, 주민기	124	14
5	윤지상, 정준홍	125	15
6	이민원, 이용수	126	16
7	김정호, 이환수	127	17
8	김태욱, 안정수	128	18

Result of the Practice

- Submit your code and terminal screen.
- Check TA's screen to see if your results are correct!

```
[Chapter2 Success!]
-['group0', 'group1', 'group2', 'group5']

[Chapter3 Success!]
-['group0', 'group4', 'group6']
```

Copyright © AI Lab. All rights reserved.

Chapter 3

Receiving and Transmitting CAN messages in CAN bus (2)

Copyright © AI Lab. All rights reserved.

Chapter 3

Backgrounds

- CAN communication practice based on Chapter1.
- Creating CAN bus and receive & transmit CAN data.

Notice

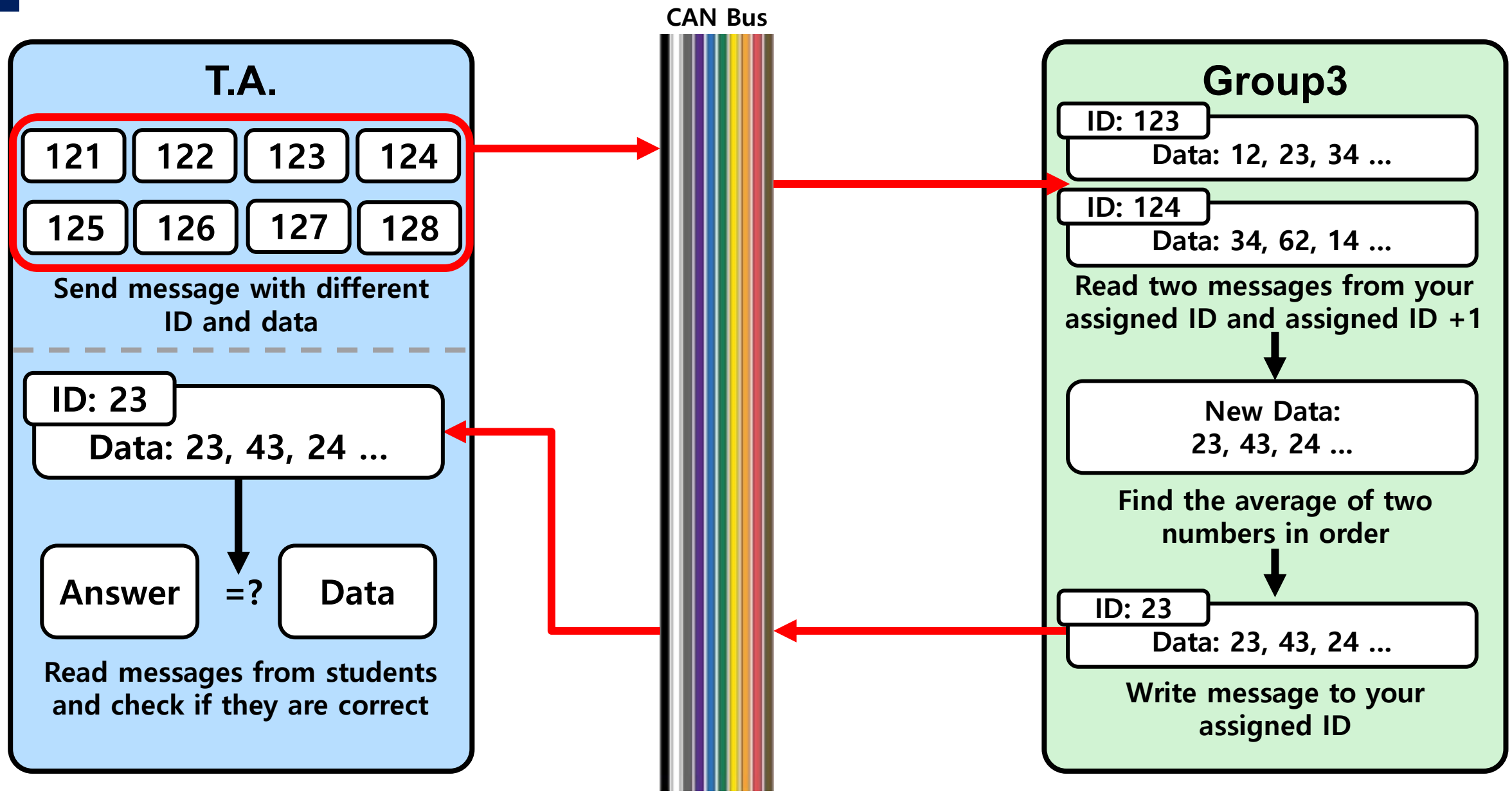
- You must have finished Chapter1 and Chapter2 to do this practice.
- Use team Orin and Kvaser device for this practice.
- Use Chapter2 code to solve the problem

Goal

- Receive CAN data from T.A. and transmit your own data back to T.A.

Copyright © AI Lab. All rights reserved.

Architecture

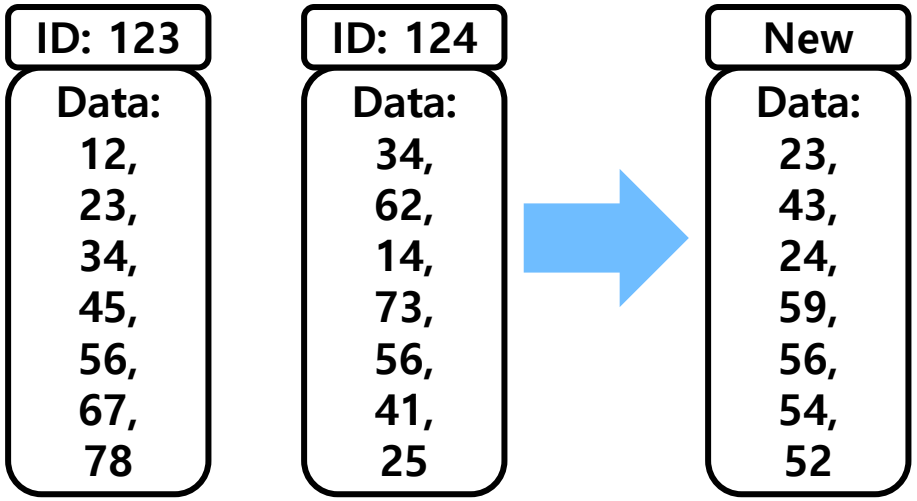


Copyright © AI Lab. All rights reserved.

Mission

- **Read the two messages assigned to you**
  - assigned ID is on the back page

- **In two messages, Find the average of two numbers in order**
  - Round off decimals



- **Send a CAN Message with the new data to the ID assigned to you**

Copyright © AI Lab. All rights reserved.

Message ID

- **Receive and Transmit CAN data!**

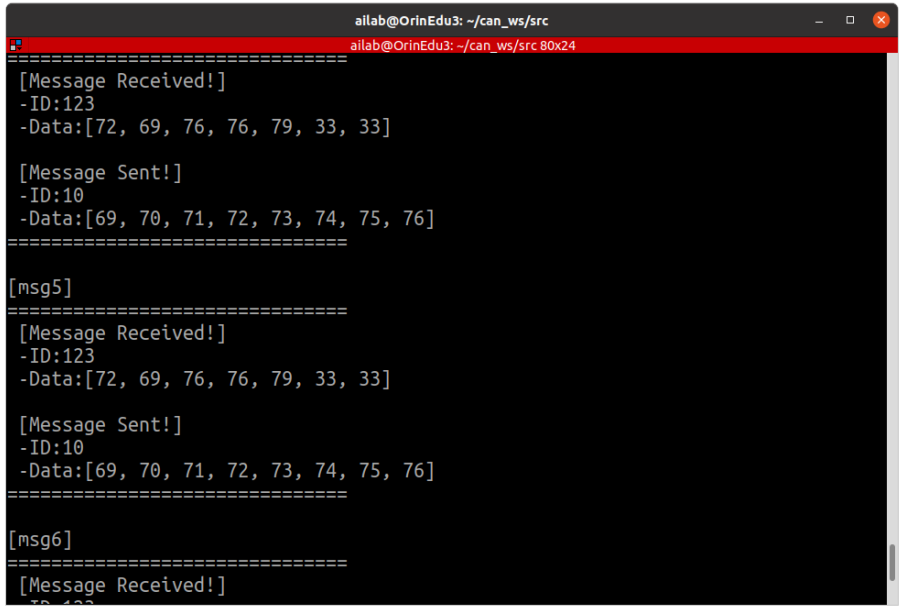
Group	Group Members	Read	Write
1	김정태, 임정기, 김동진	121, 122	21
2	박수빈, 정다훈	122, 123	22
3	허승우, 김석재	123, 124	23
4	김다혜, 주민기	124, 125	24
5	윤지상, 정준홍	125, 126	25
6	이민원, 이용수	126, 127	26
7	김정호, 이환수	127, 128	27
8	김태욱, 안정수	128, 129	28

Copyright © AI Lab. All rights reserved.

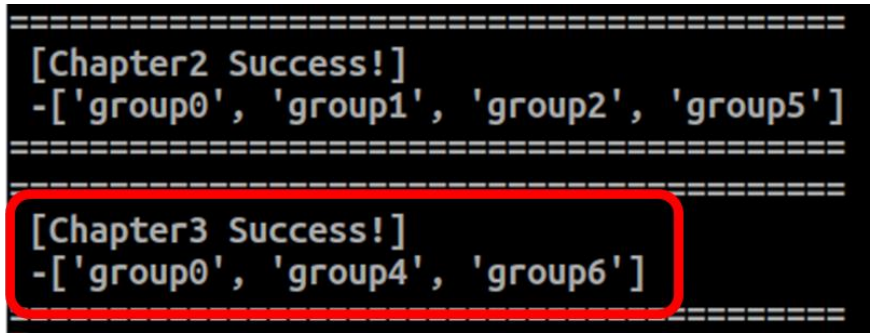
Chapter 3

Result of the Practice

- Submit your code and terminal.



- Check TA's screen to see if your results are correct!



Copyright © AI Lab. All rights reserved.



THANK YOU
FOR YOUR ATTENTION